THE NEW STACK AI Agents That Self-Initiate
Initiate Explore how AI agents move from retrieval-and-answer to proactive, self-initiated action. Register Now

AGENTIC AI

Coding Agents Don't Need Bigger Context Windows — They Need a Context Compiler

Most coding agents waste 70% of their context window on irrelevant code. A compiler would never do that. It resolves dependencies, discards unreachable code, and keeps only the interfaces where implementation isn't needed. Coding agents do the opposite—and it's costing them the exact thing a shrinking context budget can't afford to lose.

Emmimal P Alexander
August 1, 2026 • 13 min read

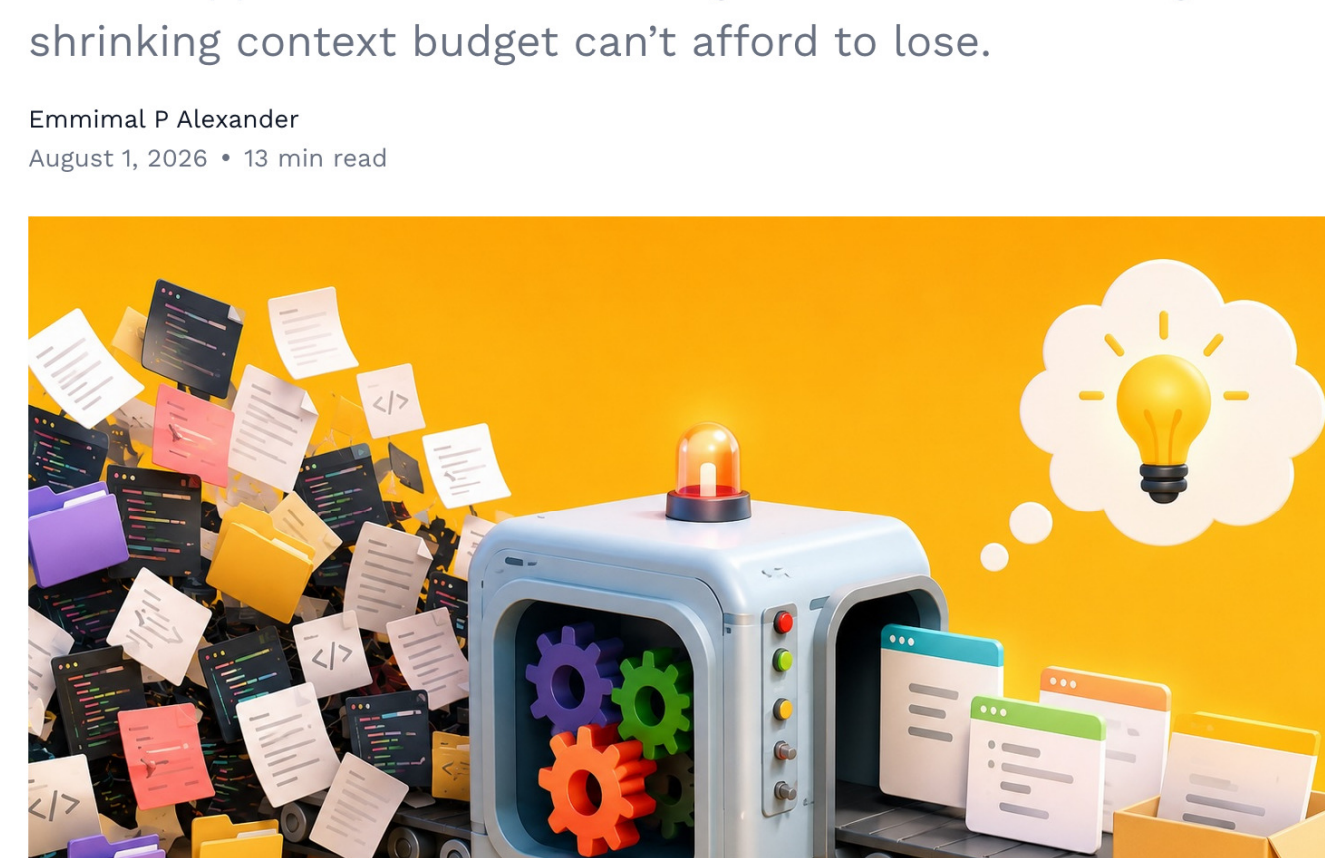


Image by the author, generated with ChatGPT (DALLE)

TL;DR:

I built a three-pass Context Compiler in pure Python. Before sending anything to the model, it figures out what your target file actually depends on, trims non-essential code down to pure interfaces, and drops everything unreachable.

Tested it on two real Python repos: it cut prompt sizes by 69–74% and ran in under 75 ms.

All the numbers below are pulled straight from captured terminal runs. If the tool couldn't determine something with certainty, it explicitly marked it instead of guessing.

Compilers Don't Just Compile Code

Compilers are just filters that know what to keep. You give one an entry point and a codebase, it traces what actually gets called, throws out the dead weight, and outputs an intermediate representation with only the detail the next step needs. Nothing extra.

Learn this step by step with the interactive **AI Agents** roadmap.

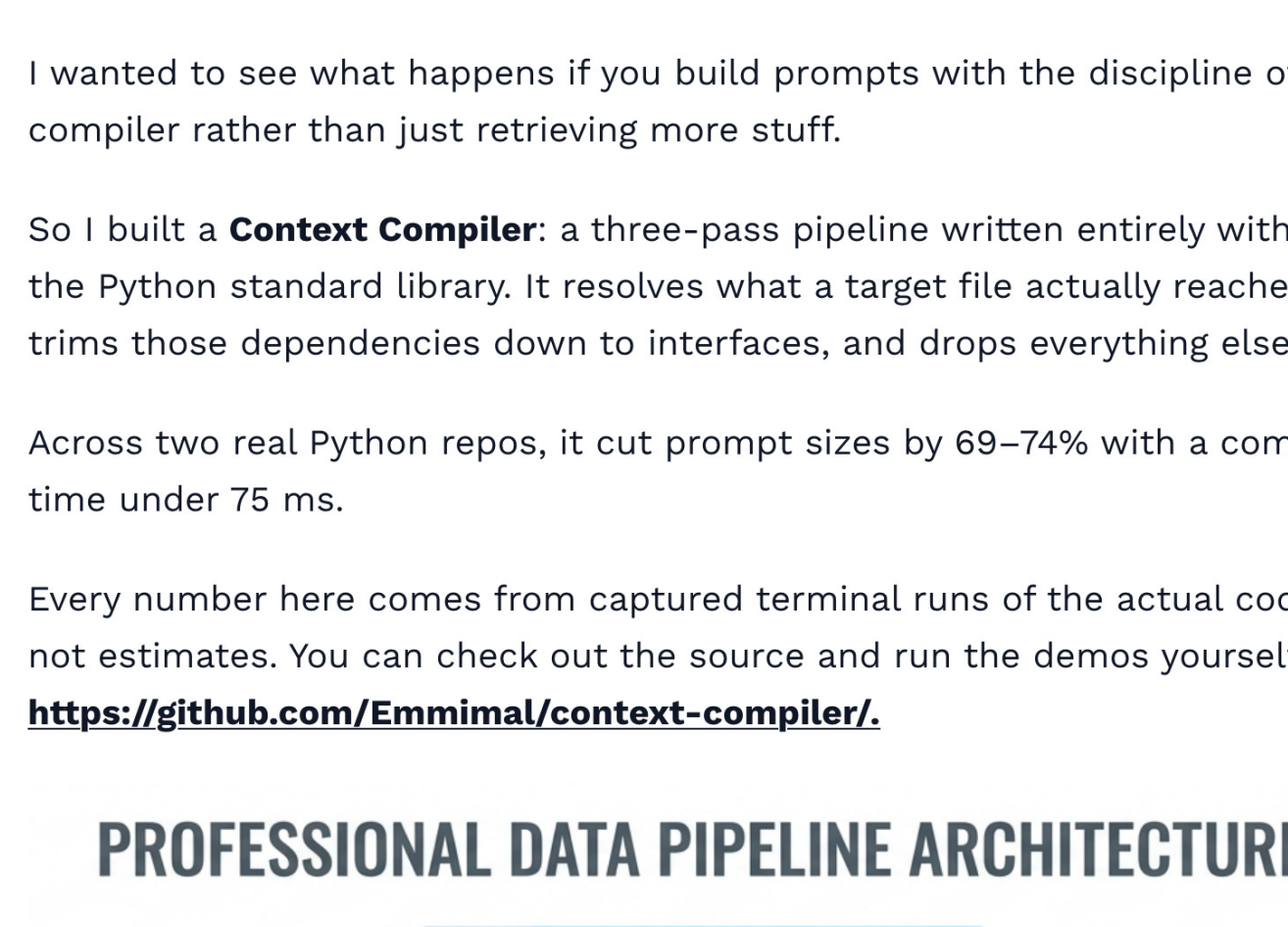
Most coding agents don't treat prompt construction like a compiler. They build some form of repository map, gather files they believe are relevant, and send them to the model with relatively little structural reduction. Larger context windows help, but they don't remove irrelevant context. That extra context competes for attention with the code that actually matters. And when window sizes shrink, that bloat triggers compaction. Compaction sounds like routine cleanup until it happens mid-task: the agent summarizes its own context to make room, then spends the next few turns trying to reconstruct implementation details from a lossy summary of a summary. Half the time an agent "forgets" something, it is just its own memory management degrading its recall.

I wanted to see what happens if you build prompts with the discipline of a compiler rather than just retrieving more stuff.

So I built a **Context Compiler**: a three-pass pipeline written entirely with the Python standard library. It resolves what a target file actually reaches, trims those dependencies down to interfaces, and drops everything else.

Across two real Python repos, it cut prompt sizes by 69–74% with a compile time under 75 ms.

Every number here comes from captured terminal runs of the actual code, not estimates. You can check out the source and run the demos yourself on **https://github.com/Emmimal/context-compiler/.**



The whole pipeline in one figure: each pass narrows the repository down by one more degree before anything reaches the model. Image by the author, generated with gemini

A quick note on timing: on July 18, OpenAI quietly dropped the default context window for Codex models from 372k down to 272k tokens. Billing correction or capability trim, it points to the same reality: you don't own the context window size. You only control what you feed into it.

Context Compilation Is Not Context Engineering, Just Once

In 2025, Andrej Karpathy coined "context engineering" to describe the overall work of deciding what goes into a model's prompt. Context compilation is just one specific application of that concept for source code. Given a known codebase and a file you want to edit, it figures out which files that edit actually relies on, then shrinks everything else down to the smallest usable form. That is as far as this project goes. I am assuming you already know how context management differs from basic prompt engineering or retrieval, so I won't rehash all of that here. Let's get right into how the compiler actually works.

Pass 1: Symbol Resolution

Pass 1 answers one basic question: starting from the file you are editing, what else in the repo actually matters? It traces explicit imports first. If a call like `.save()` cannot be explained by an import, it falls back to checking a repo-wide symbol table, expanding outward breadth-first up to a set hop limit.

Here is the exact output captured from a run against a small test repository designed to hit edge cases:

```markup
Target: app\views.py
Reachable files (3):
  hop 1: app\services.py
  hop 1: app\models.py
  hop 1: app\models_order.py

Dynamic dispatch flagged in: [WindowsPath('app/views.py')]
Event-decorator hints flagged in: {WindowsPath('app/services.py'): {'receiver'}}
Name collisions: {'save': [WindowsPath('app/models.py'), WindowsPath('app/models_order.
  app\handlers\handler_email.py: MISSED (as expected) -- getattr() dispatch is invisibl
  app\handlers\handler_sms.py: MISSED (as expected) -- getattr() dispatch is invisible
```

`services.py` and `models.py` were pulled in through explicit imports. `models_order.py` came in via bare-name resolution because `user.save()` and `Order.save()` share a method name that a simple resolver cannot separate without a type checker.

Notice what was left out: the two handler files. They are only invoked via `getattr()` at runtime, which static analysis cannot see. Excluding them is the correct behavior. A tool that makes a wild guess here passes an incorrect dependency graph to the model. Reporting an unknown is always better than making things up.

Two mechanisms drive this output:

1. **A standard module index.** Every Python file maps to its importable dotted path (like `app/models.py` becoming `app.models`). Importing resolves through a fast dictionary lookup instead of guessing against the filesystem.
2. **A symbol map fallback.** When an import does not explain a call, the compiler checks a table of every function and class definition across the repo to find where it lives.

Traversing this is strictly breadth-first. Everything discovered at hop 1 gets parsed for its own calls to build the hop 2 frontier. Files are tracked so each one is visited once, stopping when the hop limit is reached or there is nothing left to scan.

Pass 2: Interface Extraction

Pass 2 handles files that are reachable but are not the file you are editing. It strips them down to bare interfaces, keeping function signatures and docstrings while replacing every function body with a single placeholder.

Here is a real before-and-after output from a run:

```markup
Original: 448 chars | Skeleton: 173 chars | 1 function body stripped

class PaymentProcessor:
    """Handles payment capture."""

    def charge(self, amount, currency='USD'):
        """Charge a card for `amount` in `currency`."""
        ...
```

That cuts this class down by 61% while leaving the signature, default values, and docstrings intact.

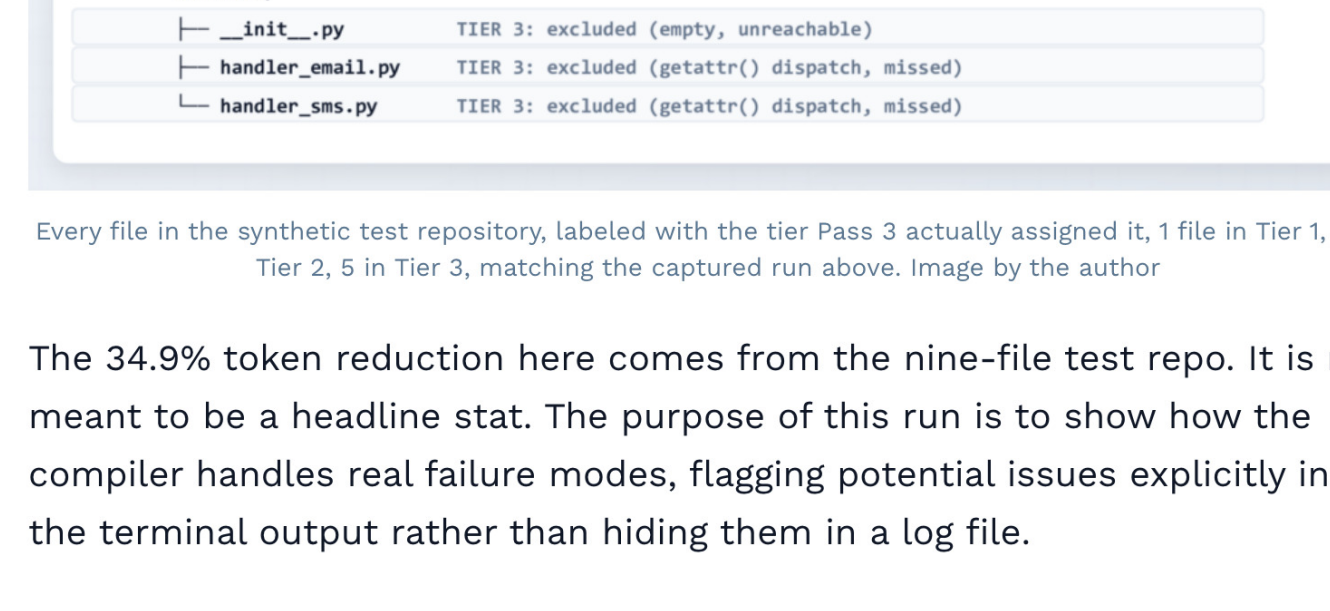
When an agent edits a file that calls `charge()`, it needs to know the method exists and what arguments it expects. It does not need the internal implementation logic. Spending context window budget on those details for every dependency is precisely what this pass cuts out.

Pass 3: Context Assembly

Pass 3 takes the output from the first two passes, builds a three-tier context, and reports the exact token cost:

```markup
Target file: C:\Users\Admin\AppData\Local\Temp\context_compiler_demo_1jspidad\app\views
Repo files scanned: 9
Tier 1 (full source): 1 file
Tier 2 (skeletonized): 3 files
Tier 3 (excluded): 5 files
Naive full-dump estimate: 556 tokens
Compiled context: 362 tokens
Reduction: 34.9%
Build time: 12.45 ms
Warning: 1 file(s) use getattr()-based dynamic dispatch — targets may be missing from t
Warning: 1 file(s) use event-style decorators (e.g. @receiver) — handlers may be missin
Note: 1 call name(s) resolved to more than one file (name-only resolution) — tier 2 may
```

Mapped onto the actual repository, those three tiers look like this:



CONTEXT ASSEMBLY PIPELINE

```
repo/
└─ app/
   ├─ __init__.py              TIER 3: excluded (empty, unreachable)
   ├─ views.py              <- TIER 1: full source (being edited)
   ├─ models.py             <- TIER 2: skeleton (reachable, hop 1)
   ├─ models_order.py       <- TIER 2: skeleton (name-collision pull-in)
   ├─ services.py           <- TIER 2: skeleton (reachable, hop 1)
   ├─ utils.py                 TIER 3: excluded (unreachable)
   └─ handlers/
      ├─ __init__.py           TIER 3: excluded (empty, unreachable)
      ├─ handler_email.py      TIER 3: excluded (getattr() dispatch, missed)
      └─ handler_sms.py        TIER 3: excluded (getattr() dispatch, missed)
```

Every file in the synthetic test repository, labeled with the tier Pass 3 actually assigned it, 1 file in Tier 1, 3 in Tier 2, 5 in Tier 3, matching the captured run above. Image by the author

The 34.9% token reduction here comes from the nine-file test repo. It is not meant to be a headline stat. The purpose of this run is to show how the compiler handles real failure modes, flagging potential issues explicitly in the terminal output rather than hiding them in a log file.

The primary knob you can adjust across all three passes is `max_hops`, which controls how far Pass 1 expands.

max_hops	What reaches Tier 2	Trade-off
1	Direct imports and calls only	Smallest payload, but a secondary helper function might be missed.
2	Direct dependencies plus one layer out	Balanced default used for all benchmark runs below.
3+	Wider transitive neighborhood	Captures a larger call graph, but savings diminish as depth grows.

All benchmarks in the next section were run with `max_hops=2`. This should be tuned based on the task: keep it wider when exploring unfamiliar code, and tighten it down once you know the local dependency structure.

Measuring the Compiler

Here are three separate benchmarks with three distinct purposes, all pulled directly from captured terminal output rather than calculated estimates:

Repository	Purpose	Files	Naive tokens	Compiled tokens	Reduction	Build time
Synthetic test repo	Verify edge cases explicitly	9	556	362	34.9%	N/A
context-compiler (self)	Reproducibility	7	9,379	2,867	69.4%	49 ms
loop-engine (external)	Test generalization	12	13,254	3,404	74.3%	66 ms

The self-benchmark exists purely for reproducibility. You can clone the repo, run `benchmark.py`, and hit that exact 69.4% figure yourself without taking my word for it.

The `loop-engine` run is what actually shows generalization. It is an external project I did not touch while building the compiler, yet the reduction remained in the same 70% ballpark despite a completely different import structure.

Test environment for both real repo runs: Python 3.12, CPU only, Windows 11, standard library only, with `max_hops=2`. End-to-end compile times ranged from roughly 43 to 73 milliseconds across repeated runs on both repositories, using benchmark.py directly. Token counts use a standard `characters // 4` estimate, so focus on the relative percentages rather than the exact token numbers.

To be clear: a naive full-repo dump is an upper bound, not necessarily what modern agent tools do. Features like Aider's repo map, Cursor's context selection, and Claude Code already perform some form of file filtering.

The more accurate comparison is against a flat repo map that skeletonizes every file without checking reachability. That is where three-tier assembly makes a difference. A flat repo map still pays a token cost for every single file in the workspace. This compiler pays zero for anything the resolver marks as unreachable.

In the `loop-engine` run, 9 out of 12 files were excluded entirely rather than skeletonized. A flat repo map cannot close that gap.

I have not benchmarked this on massive monorepos with hundreds of files, so I am not making claims about performance at that scale. Multi-file Python projects in the tens-of-files range represent the actual tested scope here.

Who Should Actually Reach for This

Now that the numbers are out of the way, here is where this tool actually makes sense to use.

This is worth adding to your setup if:

- You run multi-file agent tasks where most of the repo is irrelevant to the active edit.
- You need to stay strictly within a tight context budget.
- Your agent needs to know that external methods exist without burning tokens on their full implementations.

Skip it for single-file scripts. A simple file dump is already cheap enough that compilation does not matter there.

Skip it for codebases that rely heavily on dynamic dispatch or event buses without static registries. That is where static analysis falls short.

And skip it, at least right now, if your repository is so large that building the module index on every run causes a noticeable bottleneck. I break down that constraint in the trade-offs section below rather than ignoring it.

A simple sanity check: if your agent failed recently because it got overwhelmed by unrelated code in its context window, this layer directly solves that problem. If it failed because of vague instructions, this will not help you. That is still a prompt engineering problem.

Where Compilation Fails

Resolving calls by name instead of by type creates three specific blind spots, all visible in the Pass 1 output shown earlier:

- **Dynamic dispatch.** In the test repository, `dispatch_handler()` finds its target using `importlib.import_module()` and `getattr()` with runtime f-strings. The compiler excludes both destination files and flags them explicitly rather than guessing. Reporting an unknown is better than providing a wrong dependency graph.
- **Event-driven registration.** Functions decorated with patterns like `@receiver` fire at runtime without direct imports or explicit calls from the active file. The resolver flags common event-decorator names as hints rather than resolved dependencies, since static analysis cannot confirm reachability.
- **Name collisions.** Resolving by bare method name means `user.save()` and `Order.save()` look identical when searching for `.save()`. Both files get pulled into Tier 2. This does not break the output, but it inflates token count with an extra interface skeleton. That is a safe direction to fail in compared to dropping necessary code.

These trade-offs are intentional: speed and zero dependencies over a full type-aware call graph. Codebases that favor explicit imports over string-based plugin loading, and maintain explicit registries for events, make these blind spots traceable again. That is standard practice for human developers using basic reference searches, and it turns out to matter for the same reasons when the reader is an agent.

To put this concretely: if an agent traces a bug through a middleware layer that dispatches handlers by string name, it receives an explicit warning, a flagged `getattr()` call, and two excluded handler files. It does not get a correct handler by luck or a wrong handler delivered with false confidence. The output tells you or the agent precisely where static analysis stopped, so nobody spends time chasing bad context.

That is the core design choice here: an incomplete map with explicit warnings is far more useful than a complete map that is secretly wrong.

Engineering Trade-offs

Every compiler design involves trade-offs. Here is where the current implementation makes compromises:

- **Setting `max_hops=2` is empirical.** It worked well across the test repositories, but a codebase with deeper call chains might need `max_hops=3`.
- **Name-only call resolution prioritizes speed and simplicity.** It runs in microseconds and requires zero external dependencies, but it causes the name collision issues noted earlier. Adding a full type checker would fix those collisions, but it would destroy the "clone and run with standard Python" setup.
- **Token counts use `characters // 4`.** This heuristic is fine for rough estimates, but it strays on dense code. If you need exact counts, replacing it with `tiktoken` in `compiler.py` takes one line.
- **The decorator list is hardcoded, and the module index rebuilds on every run.** The event hints rely on a static list of common framework decorators, and `ModuleIndex` rescans the directory per execution. Both could be cached or made configurable later, but they were not blockers for this version.
- **Tiering is strictly binary.** You get full source for the edit target, and skeletons for every other reachable file. A hop 1 dependency and a hop 2 dependency receive identical treatment once they clear the resolver. Treating all reachable dependencies the same keeps the logic easy to reason about, even if it sacrifices some nuance at higher hop limits.

Running It Yourself

Every number in this article can be verified locally. You only need Python 3.9 or higher:

```bash
git clone https://github.com/Emmimal/context-compiler.git
cd context-compiler
python demo.py
```

That command builds the synthetic test repo used in Pass 1 and Pass 2, runs the pipeline, and finishes with a benchmark against the compiler's own codebase.

To run it against your own project:

```bash
python benchmark.py /path/to/repo /path/to/repo/target_file.py --max-hops 2
```

Pass the repo root as the first argument, and the file you are actively editing as the second. That file gets full-source treatment while everything reachable gets skeletonized. This is the exact command used to generate the benchmark results above.

What's Next

A type-aware resolver would fix the name-collision issue. A cached module index would speed things up on larger repos. Swapping in a true tokenizer would give exact numbers instead of estimates. None of these additions change the underlying premise, they just extend it.

Compilers do not exist to make programs shorter. They make them executable by isolating what the next stage actually requires. A context compiler does not make a codebase smaller, but it makes passing that code to an LLM practical by filtering for what actually belongs in the prompt.

Context limits will keep shifting on vendor schedules, sometimes growing and occasionally shrinking. Relying on compiler discipline instead of context window size protects your workflow regardless of what limits an API exposes. That design principle will outlast any single benchmark figure in this post.

> ***Source code, runnable demos, and the CLI are available on https://github.com/Emmimal/context-compiler/***

References

[1] GitHub, openai/codex, PR #33972 "Backport refreshed bundled model metadata to 0.144" (merged July 18, 2026). https://github.com/openai/codex/pull/33972

[2] Karpathy, A. (2025). Context Engineering. https://x.com/karpathy/status/1937902205765607626

[3] OpenAI. (2023). tiktoken: Fast BPE tokeniser for use with OpenAI's models. https://github.com/openai/tiktoken

[4] Aho, A., Lam, M., Sethi, R., & Ullman, J. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley, source of the pass-based, tiered intermediate-representation framing used throughout this article.

Disclosure

All code in this article is original work, written and tested on Python 3.12, on Windows 11 and Linux. Every benchmark and terminal output shown is from a real, captured run of `demo.py` or `benchmark.py`, reproducible by cloning the repository linked above; nothing here is calculated or estimated except where explicitly marked as a heuristic. I have no financial relationship with OpenAI, Anthropic, Cursor, Aider, or any other tool mentioned in this article.

WRITTEN BY

Emmimal P Alexander

I'm Emmimal P Alexander, an independent AI engineer and technical writer specializing in practical deep learning, neuro-symbolic systems, and model reliability. I am the author of Neural Networks and Deep Learning with Python: A...

See all from Emmimal P Alexander

AI coding assistants Context Engineering Context Window Llm Prompt Engineering

Share This Article

Towards Data Science is a community publication. Submit your insights to reach our global audience and earn through the TDS Author Payment Program.

Write for TDS

Related Articles